

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-a55ed59bea0f62157.jsonl`
- SHA-256: `c2fd2390d8b88cf6cc08ee2b55e45eb492fd326bbd88f00a2a972d00a7cd5ffd`
- Source modified: `2026-06-13T19:13:25+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:09:22.949Z)

Review backend/config/models.py FusionConfig + its registration on IndexingConfig, and backend/pipeline/segmenters/__init__.py. Verify: defaults reproduce control; pydantic types (Literal, Optional[list[list[float]]], dict[str,bool]) are valid; the `config.get('indexing')` dict-shim path exposes fusion correctly to the segmenter; unknown fusion keys don't crash; no name clash; FusionSegmenter registers without breaking cls(None,{}) probe. Find integration bugs.

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:25.922Z)

I'll review the fusion P2 changes. Let me start by reading the relevant files and the diffs.

3. user (2026-06-13T19:09:27.132Z)

On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:

- (use "git add <file>..." to update what will be committed)
- (use "git restore <file>..." to discard changes in working directory)
- modified: backend/config/models.py
- modified: backend/pipeline/detectors/person_detector.py
- modified: backend/pipeline/segmenters/__init__.py
- modified: backend/pipeline/segmenters/trajectory_hybrid.py
- modified: docs/README.md
- modified: tests/test_config.py
- modified: tests/test_trajectory_hybrid_equivalence.py

Untracked files:

- (use "git add <file>..." to include in what will be committed)
- artifacts/
- backend/pipeline/detectors/fusion_features.py

```
backend/pipeline/segmenters/fusion_hybrid.py
docs/QUALITY_ITERATIONS.md
scratch/HIGH_RISK_REMEDIATION_PLAN.md
scratch/PR1_BODY.md
scratch/PR1_HANDOFF.md
scratch/PR2_HANDOFF.md
scratch/PR2_deferred_hardening_status.patch
scratch/PR88_FIX.md
scratch/arial.ttf
scratch/debug_wsl.py
scratch/keep_awake.ps1
scratch/probe_cv2_count.py
scratch/probe_gemini.py
scratch/repro_drawtext_font.py
scratch/repro_movie_overlay.py
scratch/run_gx010125.py
scratch/stitch_min5.py
scratch/trim_video.py
tests/test_fusion_features.py
tests/test_fusion_hybrid.py
```

no changes added to commit (use "git add" and/or "git commit -a")

4. user (2026-06-13T19:09:27.842Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
```

```

40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
73         # resumes.
74         keep_frames: bool = False
75         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
76         # disabled.
77         min_free_gb: float = 0.0
78
79     class FusionConfig(BaseModel):
80         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
81         P2).
82
83         Every default reproduces control behaviour: the fusion segmenter persists the
84         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
85         is
86         true ? the person-cue features, but it does NOT gate the served handoff unless a
87         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
88         mask
89         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
90         only);
91         supplied ? off-court persons (spectators / adjacent courts) are excluded.
92         """
93         # Which trajectory substrate seeds the windows (shuttle stays primary).
94         substrate: Literal["wasb", "tracknet"] = "wasb"
95         # Windowing velocity threshold for the fusion family (the engine reads
96         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
97         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
98         velocity_threshold: float = 0.02
99         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
100        # enabled ? so the default path never imports torch / touches the GPU.
101        cue_flags: dict[str, bool] = Field(default_factory=dict)

```

```

97         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99         min_crossings: int = Field(default=0, ge=0)
100        # Served Gate B: when the person cue is on, drop windows with median in-court
players
101        # < this. 0 = OFF.
102        min_players: int = Field(default=0, ge=0)
103        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104        court_polygon: Optional[list[list[float]]] = None
105        # Net line for the person two-sided-motion split (person features only ? the shuttle
106        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107        net_axis: Literal["x", "y"] = "y"
108        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111    class IndexingConfig(BaseModel):
112        default_segmenter: str = "yolo_hybrid"
113        use_gpu: bool = True
114        motion_threshold: float = 0.08
115        min_segment_duration: float = 3.0
116        dead_time_merge_gap: float = 2.0
117        chunk_duration: float = 90.0
118        chunk_overlap: float = 10.0
119        detector_model: str = "fasterrcnn_resnet50_fpn"
120        detector_score_threshold: float = 0.5
121        yolo_velocity_threshold: float = 0.15
122        yolo_frame_skip: int = 3
123        yolo_tracker: str = "bytetrack.yaml"
124        yolo_prefetch_workers: int = 2
125        min_action_window_duration: float = 2.0
126        log_yolo_candidates: bool = True
127        store_yolo_candidates: bool = True
128        ai_handoff_padding: float = 2.0
129        ai_max_workers: int = 5
130        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
131        # chunks of a high-bitrate (4K) source blow past the provider upload cap
132        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
133        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
134        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135        # Matches gemini_refine's --clip-scale-width default.
136        ai_chunk_scale_width: int = 1280
137        # Optional debug knob: trim every video to the first N seconds before processing.
138        debug_duration: Optional[float] = None
139
140        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142        fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144        @field_validator("default_segmenter")
145        @classmethod
146        def segmenter_must_be_registered(cls, v: str) -> str:
147            # Registry check happens at runtime in main/segmenter selection.
148            # Here we just allow any string for forward compatibility.
149            return v
150
151        @model_validator(mode="after")
152        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
153            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
154            # step makes `while t < duration: t += step` loop forever, growing memory before
any
155            # GPU work. Reject the bad config at load time instead.
156            if self.chunk_overlap >= self.chunk_duration:

```

```

157         raise ValueError(
158             f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
159         "
160             f"({self.chunk_duration}); otherwise chunking never advances."
161         )
162         return self
163
164     class OutputConfig(BaseModel):
165         default_highlights_name: str = "highlights.mp4"
166         db_name: str = "sports_indexer.db"
167         # Directory where the DB, highlight reels, and processed clips are written.
168         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
169         output_dir: str = "output"
170
171     class WatermarkConfig(BaseModel):
172         """Branding overlay burned into each highlight clip during the per-clip re-encode
173         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
174         (under `stitching.watermark` in config.json) to turn it off. The text is fully
175         configurable. The concat stage stays stream-copy (-c copy)."""
176         enabled: bool = True
177         text: str = "Generated by Khelsutra.guru"
178         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
179         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
180
181         below
182         font: str = "Sans" # fontconfig family used when font_path is empty
183         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
184         fraction of frame height
185         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
186         box: bool = True # faint backing box behind the text for legibility
187         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
188         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
189         right"
190
191     class StitchingConfig(BaseModel):
192         begin_padding: float = 0.5
193         end_padding: float = 0.5
194         use_cache: bool = False
195         min_shot_count: int = 1
196         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
197
198     class SecurityConfig(BaseModel):
199         # When set, /api/process video_path must resolve under this directory (hosted/tenant
200         # mode). Default None preserves the single-user local 'any local file' workflow.
201         allowed_video_root: Optional[str] = None
202
203         # --- Chunked upload (B2, PR-2) ---
204         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
205         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
206         # contain upload_dir or /api/process will reject the uploaded paths.
207         upload_dir: str = "output/uploads"
208         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
209         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
210         max_upload_mb: int = 4096
211         max_part_mb: int = 95
212         allowed_upload_extensions: List[str] = ["mp4", "mov"]
213
214     class AppConfig(BaseModel):
215         """Root configuration object.
216
217         All runtime configuration should be accessed via an instance of this class

```

```

218         (or the loader) rather than raw dict lookups.
219         """
220
221         sport: Sport = "badminton"
222         ai_provider: str = "gemini"
223         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
224         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
225         ai_model: str = "gemini-flash-latest"
226
227         validation: ValidationConfig = Field(default_factory=ValidationConfig)
228         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
229         output: OutputConfig = Field(default_factory=OutputConfig)
230         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
231         security: SecurityConfig = Field(default_factory=SecurityConfig)
232
233         # Allow extra keys during transition so existing config.json files do not break.
234         model_config = {
235             "extra": "allow",
236             "validate_assignment": True,
237         }
238
239         # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
240         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
241         # attribute access / the typed config. A nested *section* is returned as a plain
dict
242         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
243         # section model instead breaks it (a model has no .get), which bit every validator
on
244         # real runs. Scalar leaves are returned as-is.
245         def get(self, key: str, default: Any = None) -> Any:
246             _MISSING = object()
247             val: Any = _MISSING
248             if key in type(self).model_fields or hasattr(self, key):
249                 val = getattr(self, key, _MISSING)
250             if val is _MISSING:
251                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
252                 # discovered dynamically from the model fields, so a future top-level
section
253                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
254                 for fname in type(self).model_fields:
255                     section = getattr(self, fname, None)
256                     if isinstance(section, BaseModel) and hasattr(section, key):
257                         val = getattr(section, key)
258                         break
259             if val is _MISSING:
260                 return default
261             if isinstance(val, BaseModel):
262                 return val.model_dump(mode="json")
263             return val
264
265         def __getitem__(self, key: str) -> Any:
266             val = self.get(key)
267             if val is None and key not in type(self).model_fields:
268                 # Re-raise like a real dict for unknown top-level
269                 raise KeyError(key)
270             return val
271
272
273         # Convenience type alias for callers that want to annotate
274         Config = AppConfig

```

5. user (2026-06-13T19:09:28.207Z)

```

1      from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2      from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3      from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4      from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9      __all__ = [
10         "segmenter_registry",
11         "BasePlaySegmenter",
12         "MotionPlaySegmenter",
13         "GeminiPlaySegmenter",
14         "HybridYoloSegmenter",
15         "TrackNetHybridSegmenter",
16         "WasbHybridSegmenter",
17         "FusionSegmenter",
18     ]
19

```

6. assistant (2026-06-13T19:09:30.553Z)

Let me read the new files and the diffs of the edited files.

7. user (2026-06-13T19:09:31.511Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33
34     from typing import Any, Dict, List, Optional, Tuple
35

```

```

28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30     from backend.pipeline.detectors.base import DetectorRunner
31     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors import fusion_features as ff
35
36
37     class FusionSegmenter(TrajectoryHybridSegmenter):
38         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
39 B)."""
40         family = "fusion"
41         display_label = "Fusion"
42
43         def __init__(self, db: Any, config: Any):
44             super().__init__(db, config)
45             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
46 default path).
47             self._person: Optional[Any] = None
48
49         @property
50         def name(self) -> str:
51             return "fusion_hybrid"
52
53         @property
54         def description(self) -> str:
55             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
56 rallies, "
57                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
58 and the "
59                    "AI provider sets semantic boundaries.")
60
61         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
62 Any]]:
63             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
64             if substrate == "tracknet":
65                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
66                 return TrackNetRunner(cfg), {
67                     "weights_path": cfg.weights_path,
68                     "queue_length": cfg.queue_length,
69                     "fusion_substrate": "tracknet",
70                 }
71             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
72             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
73 "wasb"}
74
75         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
76 frame_width: float, video_path: str,
77 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
78             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
79 video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
81 persisted
82             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
83 estimated
84             # over the whole trajectory by rally_gate (camera-agnostic).
85             gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
86 min_crossings=0)
87             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
88
89             fcfg = idx_cfg.get("fusion", {})
90             if fcfg.get("cue_flags", {}).get("person", False):
91                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,

```



```

fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                         frame_width: float, video_path: str,
87                         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                          window_stats: List[Dict[str, Any]],
111                          idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130    segmenter_registry.register(FusionSegmenter)
131

```

8. user (2026-06-13T19:09:32.465Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this

```

```

6     module only turns those signals into a handful of **scale/translation-invariant**
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?
43            if (yi > y) != (yj > y):
44                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                if x < x_cross:
46                    inside = not inside
47            j = i
48        return inside
49
50
51    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54        if frame_width <= 0 or frame_height <= 0:
55            return (0.0, 0.0)
56        x1, _y1, x2, y2 = box
57        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61        if frame_width <= 0 or frame_height <= 0:

```

```

62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
79     List[int]:
80         """Per-frame count of in-court persons (one entry per sampled frame)."""
81         return [
82             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
83 court_polygon))
84             for _frame_idx, dets in sorted(per_frame.items())
85         ]
86
87     def _median(vals: Sequence[float]) -> float:
88         if not vals:
89             return 0.0
90         s = sorted(vals)
91         n = len(s)
92         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
101 flicker)."""
102         if not counts:
103             return 0.0
104         return sum(1 for c in counts if c >= min_players) / len(counts)
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)

```

```

124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                       frame_height: float, net: Optional[Net],
128                       court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134     """
135     if net is None or not per_frame:
136         return 0.0
137     axis, pos = net
138     both = 0
139     for _frame_idx, dets in per_frame.items():
140         near, far = False, False
141         for _tid, box in dets:
142             if not person_in_court(box, frame_width, frame_height, court_polygon):
143                 continue
144             fx, fy = _foot_norm(box, frame_width, frame_height)
145             coord = fx if axis == "x" else fy
146             if coord < pos:
147                 near = True
148             else:
149                 far = True
150             if near and far:
151                 both += 1
152     return both / len(per_frame)
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                    pad_frac: float) -> bool:
156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?"""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                               frame_width: float, frame_height: float,
166                               pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171     """
172     if not per_frame or frame_width <= 0 or frame_height <= 0:
173         return 0.0
174     aligned = 0
175     colocated = 0
176     for p in shuttle_points or []:
177         if not getattr(p, "visible", False):
178             continue
179         dets = per_frame.get(int(p.frame))
180         if dets is None:
181             # shuttle frame wasn't person-sampled ? can't judge
182             continue
183         aligned += 1
184         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
185               for _tid, box in dets):
186             colocated += 1

```

```

185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
197 shuttle_points,
198                                 frame_width: float, frame_height: float,
199                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
200 None,
201                                 net: Optional[Net] = None,
202                                 min_players: int = 2) -> Dict[str, float]:
203         """Aggregate the per-window person features from per-frame detections + the in-
204 window
205 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
206         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
207         return {
208             "players_in_court": players_in_court(counts),
209             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
210 court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }

```

9. user (2026-06-13T19:09:33.617Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
b/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
index b2847c9..149282e 100644
```

```
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
```

```
    except (ValueError, TypeError):
```

```
        return max(3, int(duration / 0.8))
```

```
+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
```

```
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
```

```
+                             frame_width: float, video_path: str,
```

```
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
```

```
+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
```

```
+     returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
```

```
+     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
```

```
+     features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
```

```
+     return {
```

```
+         "peak_velocity": cw["peak_velocity"],
```

```
+         "mean_velocity": cw["mean_velocity"],
```

```
+         "active_frame_count": cw["active_frame_count"],
```

```
+         "track_id_count": cw["track_id_count"],
```

```
+     }
```

```
+ def _select_for_handoff(self, windows: List[Dict[str, Any]],
```

```
+                             window_stats: List[Dict[str, Any]],
```

```
+                             idx_cfg: Dict[str, Any]) -> List[int]:
```

```
+     """Indices of the candidate windows that proceed to the AI handoff (and thus may
```

```

+         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+         Persisted candidates are NOT affected ? they remain the full superset for offline
+         re-scoring."""
+         return list(range(len(windows)))
+
+     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                       player_pool: Optional[List[str]] = None,
+                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
+         f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+     # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+     # persistence and the handoff gate below.
+     window_stats = [
+         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+         for cw in all_candidate_windows
+     ]
+
+     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+     if store_candidates:
+         for i, cw in enumerate(all_candidate_windows):
+             stats = {
+                 "peak_velocity": cw["peak_velocity"],
+                 "mean_velocity": cw["mean_velocity"],
+                 "active_frame_count": cw["active_frame_count"],
+                 "track_id_count": cw["track_id_count"],
+             }
+             candidate_ids[i] = self.db.add_candidate_segment(
+                 video_id, detector=self.name,
+                 start_time=cw["start"], end_time=cw["end"],
+                 chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                 stats=stats, detection_params=detection_params,
+                 stats=window_stats[i], detection_params=detection_params,
+             )

+     if not all_candidate_windows:
+         return []

+     # --- Phase 3: AI provider handoff over padded candidate clips ---
+     # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+     # candidates above stay the full superset ? only the served play_segments are gated.
+     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
+     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+     os.makedirs(handoff_dir, exist_ok=True)
+     logger.info("Phase 3: %s validation on %d candidate windows...",
+                 provider_name, len(all_candidate_windows))
+     provider_name, len(handoff_indices))

+     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+         w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

+     ai_segments: List[dict] = []
+     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
+         futures = [executor.submit(process_candidate_window, wi, cw)
+                     for wi, cw in enumerate(all_candidate_windows)]
+         futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                     for wi in handoff_indices]

```

```

        for future in futures:
            ai_segments.extend(future.result())
    try:

```

10. user (2026-06-13T19:09:34.007Z)

```

diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                           frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's ``.frame`` exactly ? that alignment is what
+     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break
+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+             frame_idx += 1
+     cap.release()
+     return {"frames": frames, "fps": fps,
+           "frame_width": float(frame_width), "frame_height": float(frame_height)}

```

11. assistant (2026-06-13T19:09:37.234Z)

Now let me read the base segmenter and trajectory_hybrid to understand the engine, registry probe (`cls(None, {})`), and the `idx\_cfg` shape passed to these hooks.

12. user (2026-06-13T19:09:39.564Z)

```
1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self.segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self.segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self.segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self.segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
```



```

58         except Exception:
59             desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

13. user (2026-06-13T19:09:39.568Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""
53
54        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55        Any]]:

```

```

54         """Return (runner, detector-specific detection_params extras)."""
55         raise NotImplementedError
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---

```

```

114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()

```

```

176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )
191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label} rally]
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         fusion
205         # segmenter adds net_crossings + person-cue features. Computed once, reused for
206         both
207         # persistence and the handoff gate below.
208         window_stats = [
209             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210             idx_cfg)
211             for cw in all_candidate_windows
212         ]
213
214         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216         if store_candidates:
217             for i, cw in enumerate(all_candidate_windows):
218                 candidate_ids[i] = self.db.add_candidate_segment(
219                     video_id, detector=self.name,
220                     start_time=cw["start"], end_time=cw["end"],
221                     chunk_index=cw["chunk_index"], confidence=min(1.0,
222                     cw["peak_velocity"]),
223                     stats=window_stats[i], detection_params=detection_params,
224                 )
225
226         if not all_candidate_windows:
227             return []
228
229         # --- Phase 3: AI provider handoff over padded candidate clips ---
230         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
231         Persisted
232         # candidates above stay the full superset ? only the served play_segments are
233         gated.
234         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
235         idx_cfg)
236
237         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
238         os.makedirs(handoff_dir, exist_ok=True)
239         logger.info("Phase 3: %s validation on %d candidate windows...",
240                     provider_name, len(handoff_indices))
241

```

```

231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })

```

```

289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

14. assistant (2026-06-13T19:09:44.434Z)

Now I understand the registry probe. ``cls(None, {})`` calls ``FusionSegmenter.__init__(None, {})`` which calls ``super().__init__(db, config)`` ? ``BasePlaySegmenter.__init__`` sets ``self.config = {}``. Then ``name`` is accessed (a plain property returning a string), which is safe.

Let me verify a few things: the ``idx_cfg`` shape (it's ``config.get("indexing", {})`` which via the shim returns a dict from ``model_dump``), the ``WasbConfig.from_indexing_cfg`` signature, and check the ``PersonDetector`` constructor and methods used.

15. user (2026-06-13T19:09:45.872Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21    (ADR-005; ultralytics' AGPL YOLO was dropped).
22    """
23    import logging
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27    import torch
28
29    from backend.utils.geometry import get_velocity_normalised
30
31    logger = logging.getLogger(__name__)
32
33    # torchvision detection models are trained on COCO where the "person" category is
34    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36    _PERSON_CLASS_ID = 1
37
38    # Permissively-licensed torchvision detectors selectable via config
39    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40    _DETECTOR_FACTORIES = {
41        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44    }

```

```

45     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48     class PersonDetector:
49         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53         min_action_window_duration}``); the model loads lazily on first use.
54         """
55
56         def __init__(self, config: Dict[str, Any]):
57             self.config = config
58             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59             # `self.model([tensor])` inference call type-checks without a quarantine.
60             self.model: Any = None
61             self.device: Optional[str] = None
62
63         def lazy_load_model(self) -> None:
64             if self.model is not None:
65                 return
66
67             # Imported lazily so the module imports without torchvision present and so
68             # the test suite (which pre-sets self.model) never pulls in model weights.
69             from torchvision.models import detection as tv_detection
70
71             idx_cfg = self.config.get("indexing", {})
72             use_gpu = idx_cfg.get("use_gpu", True)
73             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74             if use_gpu and not torch.cuda.is_available():
75                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78             if model_name not in _DETECTOR_FACTORIES:
79                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80                 model_name = _DEFAULT_DETECTOR
81
82             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85             self.model = builder(weights="DEFAULT")
86             self.model.eval()
87             self.model.to(self.device)
88
89         def make_tracker(self, fps: float) -> Any:
90             """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92             Imported lazily so tests can mock this method without supervision installed.
93             """
94             import supervision as sv
95             frame_rate = max(1, int(round(fps)))
96             return sv.ByteTrack(frame_rate=frame_rate)
97
98         def detect_and_track(self, frame: Any, tracker: Any,
99                             score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100             """Detect persons in a frame and assign persistent track IDs.
101
102             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103             This replaces the old ultralytics ``model.track(...)`` call, which bundled
104             detection AND association: torchvision does the detection, supervision's
105             ByteTrack does the association.
106             """

```

```

107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
112 1).float()).div(255.0).to(self.device)
113         with torch.no_grad():
114             outputs = self.model([tensor])
115
116         out = outputs[0]
117         boxes = out["boxes"].detach().cpu().numpy()
118         labels = out["labels"].detach().cpu().numpy()
119         scores = out["scores"].detach().cpu().numpy()
120
121         # Keep only confident person detections.

```

16. user (2026-06-13T19:09:46.249Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool
35
36
37    def _spread(vals: Sequence[float]) -> float:
38        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39        if len(vals) < 2:
40            return 0.0
41        s = sorted(vals)
42        lo = s[max(0, int(0.05 * (len(s) - 1)))]
43        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44        return hi - lo
45
46
47    def _median(vals: Sequence[float]) -> float:

```



```

48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'/'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
60         """
61         xs = [p.x for p in points if p.visible]
62         ys = [p.y for p in points if p.visible]
63         sx, sy = _spread(xs), _spread(ys)
64         if sx >= sy:
65             return "x", _median(xs), sx
66         return "y", _median(ys), sy
67
68
69     def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70         """Count sign changes of (coord - net) across visible points, ignoring the
71         deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72         crossings = 0
73         last_side = None
74         for p in window_points:
75             if not p.visible:
76                 continue
77             v = (p.x if axis == "x" else p.y) - net
78             if abs(v) < deadband:
79                 continue
80             side = v > 0
81             if last_side is not None and side != last_side:
82                 crossings += 1
83             last_side = side
84         return crossings
85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in

```

```
112         gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

17. assistant (2026-06-13T19:09:49.708Z)

Now let me look at the `WasbConfig.from\_indexing\_cfg` and `TrackNetConfig.from\_indexing\_cfg` to verify the `idx\_cfg` dict shape they expect, and check the `\_build\_runner` substrate access path. Also let me check the test files to understand intended behavior.

18. user (2026-06-13T19:09:51.373Z)

```
backend/pipeline/detectors/tracknet_runner.py:46:     def from_indexing_cfg(cls, idx_cfg:
Dict[str, Any]) -> "TrackNetConfig":
backend/pipeline/detectors/wasb_runner.py:45:     def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
```

19. user (2026-06-13T19:09:51.810Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7      """
8      from types import SimpleNamespace
9      from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
(~500)
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26         for f in range(300, 360):           # window B frames: y == net level -> deadband -> no
crossings
27             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28         return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
```

```

41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49                 patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50                 patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51                 patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52                 patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
68         (superset)
69         assert all("net_crossings" in s for s in stats)
70         # Window A (rally) crosses many times; window B (held) zero.
71         assert stats[0]["net_crossings"] >= 2
72         assert stats[1]["net_crossings"] == 0.0
73         # Person cue OFF by default -> NONE of the person features present.
74         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
75         # The 4 base motion keys are still there.
76         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
77                     for s in stats)
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                            # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.

```

```

95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
105             return_value=fake_pd):
106             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
107             stats = _stats_calls(db)
108             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
109             assert all("net_crossings" in s for s in stats)
110             fake_pd.detections_per_frame.assert_called() # the person detector WAS used

```

20. assistant (2026-06-13T19:09:56.194Z)

Now let me check the config test and verify the dict-shim behavior with ``model_dump(mode="json")`` for the ``fusion`` section, plus check ``WasbConfig.from_indexing_cfg`` to see how it reads ``idx_cfg``. I also want to verify whether ``model_dump(mode="json")`` serializes ``court_polygon=None`` and ``cue_flags={}`` correctly, and crucially whether ``extra="allow"`` is set on ``FusionConfig`` (unknown fusion keys).

21. user (2026-06-13T19:09:57.623Z)

```

30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
33         indexing.wasb)."""
34         distro: str = "Ubuntu"
35         repo_dir: str = "~/models/WASB-SBDT"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "wasb"
38         weights_path: str = "~/models/WASB-
39         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
40         sport: str = "badminton"
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
43         keep_frames: bool = False # retain staged video + frame cache after success (debug
44         only)
45         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
46         off)
47
48     @classmethod
49     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
50         w = (idx_cfg or {}).get("wasb", {}) or {}
51         return cls(
52             distro=w.get("wsl_distro", "Ubuntu"),
53             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
54             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
55             conda_env=w.get("conda_env", "wasb"),
56             weights_path=w.get("weights_path",
57                 "~/models/WASB-
58                 SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),

```

```

62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                   f"python -c \"import torch; print('torch', torch.__version__, "
70                   f"      f'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:
74                 return False, "`wsl` not found ? Windows + WSL2 required."
75             except subprocess.TimeoutExpired:
76                 return False, "WSL healthcheck timed out."
77             out = (res.stdout or "").strip()
78             if res.returncode != 0:
79                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80             return True, out
81
82         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83             src_mnt = to_wsl_mnt_path(video_win_path)
84             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
86             {wsl_tilde_quote(dest)} && echo OK"
87             try:
88                 res = self._wsl_bash(cmd)
89             except subprocess.TimeoutExpired:
90                 logger.error("Staging video into WSL timed out.")
91                 return None
92             if res.returncode != 0 or "OK" not in (res.stdout or ""):
93                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
94 res.stdout).strip())
95                 return None
96             return dest
97
98         def _stage_infer_script(self) -> bool:
99             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
100             src_mnt = to_wsl_mnt_path(_INFER_WIN)
101             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
102             OK"
103             res = self._wsl_bash(cmd)
104             return res.returncode == 0 and "OK" in (res.stdout or "")
105
106         def run_predict(self, video_win_path: str, output_win_dir: str,
107                        video_id: Optional[str] = None) -> Optional[str]:
108             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
109
110             All WSL/cache/output artifacts are namespaced by `video_id` (the file MD5) so
111             two videos that share a filename (e.g. two `match.mp4` from different folders)
112             can never reuse each other's staged frames, resumable cache, or CSV.
113
114             # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
115             # translation: to_wsl_mnt_path passes relative paths through unchanged, so
116             # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
117             # the CSV landed inside WSL while Windows polled the repo-relative path.
118             output_win_dir = os.path.abspath(output_win_dir)
119             os.makedirs(output_win_dir, exist_ok=True)
120             # Normalize for cross-platform basename (tests use Windows paths on Linux).
121             video_win_path = str(video_win_path).replace("\\", "/")
122             base = os.path.basename(video_win_path)

```

22. user (2026-06-13T19:09:58.295Z)

diff --git a/tests/test_config.py b/tests/test_config.py

```

index ec09cd5..5519f06 100644
--- a/tests/test_config.py
+++ b/tests/test_config.py
@@ -25,6 +25,36 @@ def test_load_default_config():
     assert cfg.indexing.ai_max_workers == 5

+def test_fusion_config_defaults_reproduce_control():
+    """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
+mask."""
+    cfg = load_config("/nonexistent/path/12345/config.json")
+    f = cfg.indexing.fusion
+    assert f.substrate == "wasb"
+    assert f.cue_flags == {}          # person cue OFF
+    assert f.min_crossings == 0       # Gate A OFF
+    assert f.min_players == 0        # Gate B OFF
+    assert f.court_polygon is None    # no mask
+    assert f.velocity_threshold == 0.02
+
+def test_fusion_config_file_override():
+    with tempfile.TemporaryDirectory() as td:
+        p = os.path.join(td, "fusion.json")
+        with open(p, "w") as fh:
+            json.dump({"indexing": {"fusion": {
+                "substrate": "tracknet", "min_crossings": 2,
+                "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1], [0.5,
+0.9]],
+                }}}), fh)
+        cfg = load_config(p)
+        f = cfg.indexing.fusion
+        assert f.substrate == "tracknet"
+        assert f.min_crossings == 2
+        assert f.cue_flags == {"person": True}
+        assert len(f.court_polygon) == 3
+        # An old config without a fusion block still loads (defaults applied).
+        assert cfg.indexing.wasb.velocity_threshold == 0.02
+
+def test_file_override():
+    with tempfile.TemporaryDirectory() as td:
+        p = os.path.join(td, "override.json")

```

23. assistant (2026-06-13T19:10:01.840Z)

Now I have a clear picture. Let me verify a couple of key concerns by actually running Python (the dev container has pydantic but not torch/cv2). Let me test: (1) the config defaults & dict-shim round-trip, (2) the registry probe `cls(None, {})`, (3) unknown fusion keys handling, (4) name clash check.

24. user (2026-06-13T19:10:06.234Z)

```

defaults fusion: {'substrate': 'wasb', 'velocity_threshold': 0.02, 'cue_flags': {},
'min_crossings': 0, 'min_players': 0, 'court_polygon': None, 'net_axis': 'y', 'net_position':
0.5}
idx is dict: True
fusion in idx: True
fusion val type: <class 'dict'>
substrate via shim: wasb
cue_flags via shim: {}
court_polygon via shim: None
min_crossings via shim: 0

```

25. assistant (2026-06-13T19:10:08.765Z)

The dict-shim correctly exposes fusion as a nested dict. Now let me verify the registry probe, unknown-key handling, name clash, and Literal/Optional validation.

26. user (2026-06-13T19:10:14.717Z)

```
unknown fusion key accepted, dump: {'substrate': 'wasb', 'velocity_threshold': 0.02,
'cue_flags': {}, 'min_crossings': 0, 'min_players': 0, 'court_polygon': None, 'net_axis': 'y',
'net_position': 0.5}
AppConfig with unknown fusion key OK; fusion dump: {'substrate': 'wasb', 'velocity_threshold':
0.02, 'cue_flags': {}, 'min_crossings': 0, 'min_players': 0, 'court_polygon': None, 'net_axis':
'y', 'net_position': 0.5}
bad substrate rejected (good)
bad net_axis rejected (good)
court_polygon parsed: [[0.1, 0.1], [0.9, 0.1], [0.5, 0.9]]
```

27. assistant (2026-06-13T19:10:18.526Z)

Interesting ? unknown fusion keys are silently dropped (not stored). `FusionConfig` doesn't have `extra="allow"`, so the default `extra="ignore"` applies: unknown keys don't crash but are discarded. That matches the WASB section pattern. No crash. Now let me check the registry probe and name clash, plus run the actual test files to confirm they pass.

28. user (2026-06-13T19:10:27.703Z)

```
registered names: ['motion', 'gemini', 'yolo_hybrid', 'tracknet_hybrid', 'wasb_hybrid',
'fusion_hybrid']
any duplicate registration overwrote? fusion_hybrid present: True
probe name: fusion_hybrid
probe description ok: True
_person init: None
```

29. assistant (2026-06-13T19:10:31.645Z)

Registry probe works, no name clash. Now let me run the actual test suites for fusion and config to confirm they pass, and check `family = "fusion"` interaction. The `velocity\_thresh` in the engine reads `idx\_cfg.get(self.family, {}).get("velocity\_threshold", 0.02)` where `self.family = "fusion"`. So it reads `idx\_cfg["fusion"]["velocity\_threshold"]`. That's the FusionConfig's `velocity\_threshold`. Good ? that's intentional and works.

Let me run the test files.

30. user (2026-06-13T19:10:40.367Z)

```
..... [100%]
42 passed in 5.82s
```

31. assistant (2026-06-13T19:10:52.679Z)

All tests pass. Now let me dig into potential integration bugs that the tests might not catch. Key concerns:

1. **`\_build\_runner` reads `idx\_cfg.get("fusion", {}).get("substrate", "wasb")` but for tracknet it calls `TrackNetConfig.from\_indexing\_cfg(idx\_cfg)`** ? that reads `idx\_cfg["tracknet"]`, NOT `idx\_cfg["fusion"]`. So substrate=tracknet uses the tracknet config block, not fusion. That's defensible but worth noting. More importantly: the `velocity\_threshold` for the fusion family comes from `idx\_cfg["fusion"]["velocity\_threshold"]` (engine line 149, `self.family="fusion"`), but the runner's own windowing config... let me check `trajectory\_to\_action\_windows` uses the passed `velocity\_thresh`. Yes, line 186-190 passes `velocity\_thresh` explicitly. Good.

2. **The `net\_crossings` Gate-A computation uses `rally\_gate.gate\_windows(points, [(cw["start"], cw["end"]]), fps, min\_crossings=0)`**. Let me check the `\_points\_in` frame filtering vs how `points` frames relate. Let me look closer at a subtle issue: in `\_person\_features`, `shuttle\_in = [p for p in points if start\_f <= p.frame <= end\_f]` ? but `compute\_person\_features` receives `shuttle\_in` and uses `int(p.frame)`. Fine.

3. **Critical concern**: ``detections_per_frame`` returns ``frames`` keyed by frame index, but the ``per_frame`` type in `fusion_features` is ``Dict[int, List[Tuple[int, BBox]]]``. The colocation uses ``per_frame.get(int(p.frame))``. The person detection samples frames at ``(frame_idx - start_frame) % step == 0``. The shuttle trajectory has every frame. So alignment depends on ``start_f`` matching. Let me verify the ``mean_player_motion`` frame_rate / step issue and check the ``make_tracker`` call.

Let me check one real concern: in ``detections_per_frame``, ``score_thresh`` is read via ``self.config.get("indexing", {}).get(...)``. But ``self.config`` here is the `PersonDetector`'s config, which is ``self.config`` passed from ``FusionSegmenter`` (line 94: ``PersonDetector(self.config)``). The `FusionSegmenter`'s ``self.config`` is the `AppConfig`. Let me verify ``PersonDetector`` is given the right config type.

32. user (2026-06-13T19:10:53.829Z)

```

120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []
124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                 dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157
158         fps = cap.get(cv2.CAP_PROP_FPS)
159         if fps <= 0:
160             fps = 30.0
161
162         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163         if frame_width <= 0:
164             frame_width = 1280.0 # Sensible fallback
165

```



```

166     # Effective sampling rate after frame-skipping
167     effective_fps = fps / max(frame_skip, 1)
168
169     # A fresh tracker per chunk ? track IDs only need to be consistent within a
170     # chunk (windows are computed per chunk, then merged across chunks by time).
171     tracker = self.make_tracker(effective_fps)
172     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175     # so windows can be enriched with motion stats for logging + fine-tuning.
176     active_frames = []
177     frame_idx = 0
178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200
201             if track_id in track_history:
202                 prev_bbox = track_history[track_id]
203                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204
205                 if velocity > velocity_thresh:
206                     frame_peak_velocity = max(frame_peak_velocity, velocity)
207                     frame_active_ids.add(track_id)
208
209             track_history[track_id] = bbox
210
211         # Prune stale IDs that are no longer visible ? prevents false
212         # velocity spikes when the tracker reassigns an old ID.
213         track_history = {k: v for k, v in track_history.items() if k in current_ids}
214
215         if frame_active_ids:
216             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
217
218         frame_idx += 1
219
220     cap.release()
221
222     # Group active frames into continuous windows
223     if not active_frames:
224         return []
225
226     max_gap_frames = int(merge_gap * fps)
227
228     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
229         velocities = [v for _, v, _ in group]

```

```

229         track_ids = set().union(*[ids for _, _, ids in group])
230     return {
231         "start": group[0][0] / fps + chunk_start,
232         "end": group[-1][0] / fps + chunk_start,
233         "active_frame_count": len(group),
234         "peak_velocity": max(velocities),
235         "mean_velocity": sum(velocities) / len(velocities),
236         "track_id_count": len(track_ids),
237         "chunk_index": chunk_index,
238     }
239
240     windows = []
241     group = [active_frames[0]]
242     for af in active_frames[1:]:
243         if af[0] - group[-1][0] <= max_gap_frames:
244             group.append(af)
245         else:
246             windows.append(_finalize(group))
247             group = [af]
248     windows.append(_finalize(group))
249
250     # Filter out extremely short action windows
251     min_window_duration = self.config.get("indexing",
252     {}).get("min_action_window_duration", 2.0)
253     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254
255     return windows

```

33. assistant (2026-06-13T19:11:04.579Z)

Now I see a real concern. In `detections\_per\_frame` (the new method), the tracker is created once and `detect\_and\_track` updates it per sampled frame. But there's a subtle issue with the `players\_in\_court` / Gate B threshold comparison. Let me focus on actual integration bugs.

Let me examine the **Gate B comparison type issue** in `\_select\_for\_handoff`:

Line 124: `if person\_on and min\_players > 0 and st.get("players\_in\_court", 0.0) < min\_players:`

`players\_in\_court` returns a float median. `min\_players` is `int`. Comparing float < int is fine. But the issue: when person cue is OFF, `players\_in\_court` is never persisted, so `st.get("players\_in\_court", 0.0)` returns 0.0. But the guard requires `person\_on` to be True, so this is only reached when person is on. Good.

Now, a more important concern: **the `min\_players` config default and the `in\_court\_ratio` min_players parameter conflate**. Let me check the net_crossings value handling and whether `players\_in\_court` key matches between persistence and gate. The feature is `players\_in\_court` (line 205 in fusion_features). Gate B reads `st.get("players\_in\_court")`. Match. Good.

Let me now check a real potential bug: the `velocity\_threshold` of the fusion family vs the substrate runner. The runner is built via `WasbConfig.from\_indexing\_cfg(idx\_cfg)` which reads `idx\_cfg["wasb"]`, but the windowing `velocity\_thresh` comes from `idx\_cfg["fusion"]["velocity\_threshold"]`. Let me verify `WasbConfig` doesn't have a `velocity\_threshold` field that's used elsewhere? note the `WasbIndexConfig` (config model) HAS `velocity\_threshold: float = 0.02` (line 69) but the `WasbConfig` dataclass (runner) does NOT. Let me confirm the runner's `velocity\_threshold` isn't read anywhere.

34. user (2026-06-13T19:11:06.126Z)

No matches found

35. user (2026-06-13T19:11:06.250Z)

```

1     """Pure-logic unit tests for the person-cue fusion features
    (backend.pipeline.detectors.fusion_features).

```

```

2
3 No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path
and
4 edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5 """
6 from types import SimpleNamespace
7
8 import pytest
9
10 from backend.pipeline.detectors import fusion_features as ff
11
12
13 def _shuttle(frame, x, y, visible=True):
14     return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17 # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18 FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19 LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22 # ----- point_in_polygon
23
24 def test_point_in_polygon_inside_outside():
25     assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26     assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27     assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28     assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31 def test_point_in_polygon_degenerate():
32     assert ff.point_in_polygon((0.5, 0.5), None) is False
33     assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36 # ----- person_in_court
37
38 def test_person_in_court_uses_foot_point_and_polygon():
39     # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40     in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41     out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42     assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43     assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44     # None polygon => everyone counts
45     assert ff.person_in_court(out_box, 100, 100, None) is True
46     # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47     assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50 # ----- counts / ratios
51
52 def test_in_court_counts_and_players_and_ratio():
53     near = (10, 10, 30, 30) # foot (0.2,0.3) in LEFT_HALF
54     far = (60, 10, 80, 30) # foot (0.7,0.3) NOT in LEFT_HALF
55     per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56     # With LEFT_HALF mask: counts = [1, 1, 0, 2]
57     counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
58     assert counts == [1, 1, 0, 2]
59     assert ff.players_in_court(counts) == 1.0 # median of [0,1,1,2]
60     assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
61     assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)

```

```

62     # No mask: far counts too -> [2,1,0,2]
63     assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
64     assert ff.players_in_court([]) == 0.0
65     assert ff.in_court_ratio([]) == 0.0
66
67
68     # ----- mean_player_motion
69
70     def test_mean_player_motion():
71         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
72         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74         # single frame -> 0; no shared track id -> 0
75         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79     # ----- two_sided_motion
80
81     def test_two_sided_motion():
82         net = ("y", 0.5)
83         near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
84         far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
85         both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
86         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87         one_sided = {0: [(1, near)], 1: [(2, near)]}
88         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0      # no net -> 0
90
91
92     # ----- shuttle_player_colocation
93
94     def test_shuttle_player_colocation():
95         box = (40, 40, 60, 60)
96         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99         # aligned visible frames = {0, 1}; collocated = {0} -> 0.5
100         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101         # all-held -> 1.0
102         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109     # ----- aggregator
110
111     def test_compute_person_features_keys():
112         near = (10, 10, 30, 40)
113         far = (10, 60, 30, 90)
114         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115         shuttle = [_shuttle(0, 20, 25, True)]
116         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
court_polygon=None, net=("y", 0.5))
117
118         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
119         assert all(isinstance(v, float) for v in feats.values())
120         assert feats["two_sided_motion"] == pytest.approx(1.0)      # both sides occupied every

```



```

(debug only)
43         min_free_gb: float = 0.0                                # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

```

39. assistant (2026-06-13T19:11:38.104Z)

Important finding. Importing ``backend.pipeline.segmenters`` pulls in `**torch` and `cv2` at registry import time. But this is NOT new with this PR ? ``trajectory_hybrid`` already imports ``cv2`` at top level (line 26), and the other hybrids (``wasb_hybrid``, ``tracknet_hybrid``) already imported ``wasb_runner`/`tracknet_runner``. The dev container reportedly lacks torch/cv2 per CLAUDE.md, yet the import succeeded here ? meaning this particular environment DOES have them. So the "person path never imports torch" claim in the fusion docstring is about ``PersonDetector`/torchvision` specifically, which is lazily imported inside ``_person_features`` (line 92). Let me verify that the registry import does NOT pull torchvision/supervision (the heavy person-detector deps), since that's the actual claim.

40. user (2026-06-13T19:11:46.167Z)

```

torchvision imported by registry: False
supervision imported by registry: False
person_detector module imported: True
fusion_features imported: True

```

41. assistant (2026-06-13T19:11:58.774Z)

The claim holds: the registry import pulls ``person_detector`` module (which imports ``torch`` and ``cv2`` at top level ? but those are present in this env), but NOT ``torchvision`/`supervision`` (the heavy weights). The fusion docstring's claim "the default path never imports torch / touches the GPU" is slightly imprecise ? ``torch`` IS imported at module-load via ``person_detector.py`` top-level ``import torch`` (line 27). But the lazy import of ``from backend.pipeline.detectors.person_detector import PersonDetector`` inside ``_person_features`` is redundant since the module is already imported transitively. Not a bug, just a doc nuance.

Now let me check a genuine concern about the `**`net_crossings`` value comparison in Gate A\*\*. ``st.get("net_crossings", 0.0) < min_crossings`` ? ``net_crossings`` is a float (``float(gated[0].crossings)``), ``min_crossings`` an int. Float < int comparison fine.

Let me now verify the `**`process_video`` return type\*\* for FusionSegmenter when no API key ? it returns ``[]`` (bare list), consistent with the base. And let me check whether ``tests/test_trajectory_hybrid_equivalence.py`` confirms the twins are byte-identical (the refactor risk). Let me look at the diff of that test and the equivalence concern: the refactor moved stats construction into ``_enrich_candidate_stats`` and changed the handoff loop. The base returns the SAME 4 keys, so wasb/tracknet are unchanged. Tests passed.

Let me check one final real concern: in the handoff loop refactor, ``process_candidate_window(wi, all_candidate_windows[wi])`` is called with ``wi`` from ``handoff_indices``. Inside, ``candidate_ids[wi]`` is indexed (line 246). ``candidate_ids`` has length ``len(all_candidate_windows)``, and ``wi`` is a valid index into it. Good? the original used ``enumerate(all_candidate_windows)`` so ``wi`` was the full-list index, and ``candidate_ids[wi]`` worked. Now ``wi`` is still a full-list index (from ``handoff_indices`` = list of indices into ``all_candidate_windows``). So ``candidate_ids[wi]`` and ``all_candidate_windows[wi]`` are both correct. No off-by-one.

Let me double check the equivalence test diff to ensure nothing about the gating broke the twins, and check the README diff briefly.

42. user (2026-06-13T19:11:59.636Z)

```
warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
index 495a0cd..dfe58af 100644
--- a/tests/test_trajectory_hybrid_equivalence.py
+++ b/tests/test_trajectory_hybrid_equivalence.py
@@ -100,3 +100,25 @@ def test_base_is_not_registered():
     # The base engine itself must not appear as a selectable segmenter.
     assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
                     for cls in available.values())
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
+    """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-identical:
+    the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak in."""
+    db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
+    _, kwargs = db.add_candidate_segment.call_args
+    assert set(kwargs["stats"]) == {
+        "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"
+    }
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_base_fusion_hooks_are_identity(cls, family, detector):
+    """Directly pin the two fusion seams on the base: stats = the 4 motion keys verbatim,
+    and every window is selected for handoff (no gating)."""
+    seg = cls(MagicMock(), {"indexing": {}})
+    cw = _window()
+    stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
+    assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
+                    "active_frame_count": 30, "track_id_count": 1}
+    assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
```

43. assistant (2026-06-13T19:12:10.477Z)

Equivalence is well-pinned. Now let me reconsider potential real bugs more carefully.

****Concern: empty ``all_candidate_windows`` and ``window_stats``.**** When ``all_candidate_windows`` is empty, ``window_stats = []``, ``store_candidates`` loop does nothing, then ``if not all_candidate_windows: return []``. The ``_select_for_handoff`` is called only AFTER the early return. Good.

****Concern: ``_select_for_handoff`` calls ``int(fcfg.get("min_crossings", 0))``.**** If ``fcfg`` came from the dict-shim it's a dict with ``min_crossings`` as int. But what if someone passes config with ``min_crossings`` as a string? Pydantic would coerce on load. With the dict-shim it's always typed. But the segmenter can also be constructed with a raw dict ``{"indexing": {"fusion": {"min_crossings": "2"}}}`` (as the test does with ints). ``int("2")`` works, ``int(None)``... ``fcfg.get("min_crossings", 0)`` defaults to 0 if absent. Fine.

Now the most subtle real issue I want to verify: ****does the dict-shim correctly nest when**

`idx\_cfg` comes through `self.config.get("indexing", {})`? ** The engine line 121 does `idx\_cfg = self.config.get("indexing", {})`. If `self.config` is an `AppConfig`, the shim returns `model\_dump(mode="json")` of the IndexingConfig ? a fully nested plain dict including `fusion`. Confirmed earlier. But if `self.config` is a raw dict (as in tests and the registry probe), `idx\_cfg` is whatever was passed. Both paths work.

Let me verify one final real-world path: **a config.json with a fusion block loaded through the actual loader**, ensuring `default\_segmenter: "fusion\_hybrid"` selects the FusionSegmenter and the nested fusion dict flows. Let me check how main.py selects the segmenter and constructs it (does it pass AppConfig or a dict?).

44. user (2026-06-13T19:12:11.697Z)

```
backend/main.py:34:from backend.config import load_config as load_app_config, AppConfig,
StitchingConfig # new typed config
backend/main.py:81:# New code should import load_app_config / AppConfig from backend.config
directly.
backend/main.py:148:    default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
backend/main.py:170:        segmenter_cls = segmenter_registry.get(seg_name)
backend/main.py:272:    if req.segmenter and not segmenter_registry.get(req.segmenter):
backend/main.py:712:        seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")
backend/main.py:729:        segmenter_cls = segmenter_registry.get(seg_name)
```

45. user (2026-06-13T19:12:13.169Z)

```
backend\main.py:170:        segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:190:        segmenter = segmenter_cls(db_instance, global_config)
backend\main.py:272:    if req.segmenter and not segmenter_registry.get(req.segmenter):
backend\main.py:729:        segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:737:        segmenter = segmenter_cls(db_instance, global_config)
backend\pipeline\segmenters\base.py:42:        temp_inst = segmenter_cls(None, {})
```

46. assistant (2026-06-13T19:12:17.342Z)

`segmenter\_cls(db\_instance, global\_config)` ? `global\_config` is an `AppConfig` instance. So `FusionSegmenter.\_\_init\_\_` receives the AppConfig, passes it to `super().\_\_init\_\_` ? stored as `self.config`. Then `self.config.get("indexing", {})` uses the shim ? nested dict. Confirmed end-to-end.

Now let me verify the full test suite isn't broken by these changes and check the `run\_all\_tests.py` quickly for the segmenter list test that might assert a fixed count.

47. user (2026-06-13T19:12:28.480Z)

```
..... [ 92%]
..... [100%]
78 passed, 383 deselected in 8.14s
```

48. assistant (2026-06-13T19:12:39.808Z)

All green. Let me do a final scan for two remaining real concerns:

1. **The `min\_players` config default for Gate B vs the `min\_players=2` default in `in\_court\_ratio`/`compute\_person\_features`.** `compute\_person\_features` is called in `\_person\_features` (line 106) WITHOUT passing `min\_players`, so `in\_court\_ratio` uses default `min\_players=2`. The config's `FusionConfig.min\_players` (the served Gate B threshold, default 0) is a SEPARATE concept used only in `\_select\_for\_handoff`. These are two different `min\_players`. Not a bug, but a potential confusion ? the persisted `in\_court\_ratio` always uses 2 regardless of config `min\_players`. That's defensible (ratio is a fixed feature), worth a low note.

2. **`net` tuple in `\_person\_features`**: `net = (fcfg.get("net_axis", "y"),

`float(fcfg.get("net_position", 0.5))`). This is always a non-None tuple, so ``two_sided_motion`` always computes (never the None $\neq$ 0.0 path). Fine, since `net_axis/net_position` always have config defaults.

Let me verify there's no issue with ``court_polygon`` being passed as list-of-lists ``[[x,y]]`` from config (JSON) into ``point_in_polygon`` which expects tuples. Line 103: ``polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None`` ? converts to tuples. Good.

Let me check one edge: what if ``court_polygon`` in config is an empty list ``[]``? ``poly = []`` is falsy ? ``polygon = None`` ? mask disabled (count everyone). Reasonable. And ``point_in_polygon`` guards ``len < 3``. Good.

Final check ? verify ``detections_per_frame`` returns key ``"frames"`` (dict) and ``_person_features`` reads ``det["frames"]`` (line 98, 106). Match. And ``det["frame_width"]``/``det["frame_height"]`` keys exist. Match.

I've thoroughly verified all the requested aspects. Let me compile findings.

49. user (2026-06-13T19:13:14.416Z)

Structured output provided successfully

50. assistant (2026-06-13T19:13:25.615Z)

I completed the review. All six verification targets pass; I found no high- or medium-severity integration bugs, only three low-severity polish items. Findings are in the structured output.

Summary of what I verified (against the actual files plus live Python in this env):

- Defaults reproduce control ? ``backend/config/models.py:78-108`` (``FusionConfig``): wasb substrate, empty ``cue_flags``, ``min_crossings``/``min_players`` = 0, ``court_polygon=None``. The base ``_enrich_candidate_stats``/``_select_for_handoff`` seams in ``backend/pipeline/segmenters/trajectory_hybrid.py:82-104`` are identity, so the wasb/tracknet twins stay byte-identical (pinned by the new equivalence tests).
- Pydantic types valid ? ``Literal['wasb','tracknet']``, ``Literal['x','y']``, ``Optional[list[list[float]]]``, ``dict[str,bool]`` all instantiate and validate; bad enum values are rejected.
- Dict-shim path ? ``AppConfig.get('indexing')`` returns a fully nested plain dict (via ``model_dump(mode="json")``), so ``idx_cfg.get('fusion',{}).get(...)`` resolves; ``backend/main.py:190`` passes the ``AppConfig`` instance into ``segmenter_cls(db, global_config)``, so the production path matches.
- Unknown fusion keys ? ``FusionConfig`` uses the default ``extra="ignore"``, so unknown keys are silently dropped (no crash), matching the WASB section.
- No name clash ? registry shows ``fusion_hybrid`` distinct from the other five; ``family="fusion"`` is a fresh config section.
- Registry probe ? ``FusionSegmenter(None, {})`` succeeds; ``name``/``description`` resolve; `torchvision/supervision` stay lazy.

Three low-severity notes: the "never imports torch" docstring in ``fusion_hybrid.py:13`` overstates the guarantee (torch is already imported transitively at registry load; only `torchvision/supervision/GPU` are deferred); Gate-A re-estimates the trajectory axis/net once per window (``fusion_hybrid.py:77``) ? a perf smell, not a correctness bug; and the two distinct ``min_players`` (served Gate-B threshold vs the fixed ``in_court_ratio`` feature default of 2) could confuse a future reader.